

Homework 9

Due: Monday, December 2nd, 11pm

This worksheet contains only the *written* parts of HW9. The coding instructions will be released Thursday and be due Friday (12/06). The coding portion of this assignment is long, so you should **aim to finish the written part early (Friday night (11/29) is a good goal)** to have plenty of time to work on the code.

Submission

After completing all parts below, submit your solutions as a PDF on Gradescope under “**HW9 Written**”.

Don't forget to check that the submitted file is up-to-date with all written work you completed!

Make sure your work is legible and scanned clearly if you handwrite it, or compiled correctly if you choose to use LaTeX. Match each HW problem to the page with your work when you turn in. If your work is not readable or pages are not assigned correctly, you will receive a point deduction.

The next two pages contain the same definitions that we covered in section. Feel free to just skim/refer back to for reference when you start the tasks.

In this assignment, we will work with locations on a 2D map. For the sake of variety, rather than representing these as pairs, we will represent them as records as follows:

type Location := {x: ℝ, y: ℝ}

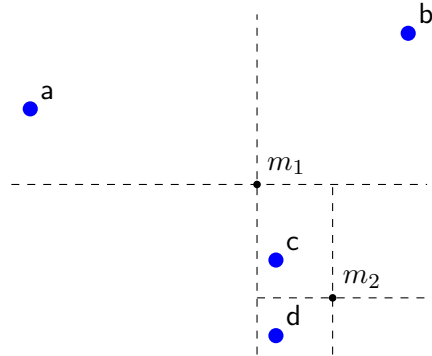
A basic operation on locations is calculating the distance between them. This can be done using the function $\text{dist} : (\text{Location}, \text{Location}) \rightarrow \mathbb{R}$, which is defined as follows:

$$\text{dist}(\ell, r) := \sqrt{(\ell.x - r.x)^2 + (\ell.y - r.y)^2}$$

We will be interested in finding the location, from amongst a list of possibilities, that is closest to a given target location. The simplest way to do this would be to calculate the distance of each location in the list to the target location. However, we can do this more efficiently by using a better data structure, namely, a tree, similar to the one we used in HW8, that recursively break up the points into smaller rectangles until each rectangle is either empty or contains a single point.

Trees of Locations

For example, with the four blue points below, we would start by splitting them at point “ m_1 ” (the centroid of those locations). The NW and NE regions then contain only a single point and the SW region is empty. The SE region, however, still contains two points, so we would split them again at point “ m_2 ”, which leaves four regions containing at most one point.



We can represent this tree with the following inductive data type:

```
type LocTree := empty
      | single(loc : Location)
      | split(at : Location, nw : LocTree, ne : LocTree, sw : LocTree, se : LocTree)
```

The constructor “empty” represents a region that is empty, while “single(s)” represents a region containing a single location at location s . The final constructor “split(m , nw, ne, sw, se)” represents a region that splits at location m into the four regions to the NW, NE, SW, and SE, respectively.

With those definitions, the example above would be represented with the following tree:

```
split( $m_1$ , single( $a$ ), single( $b$ ), empty,
      split( $m_2$ , single( $c$ ), empty, single( $d$ ), empty))
```

Regions

Note that, while each node represents some rectangular region in space, the bounds of that region are not recorded in any field. That said, we can easily calculate the bounds of the region as we work down to that node, recursively, through the tree.

Specifically, we can store a rectangular region as an instance of the following type

type Region := {x1: $\mathbb{R}$, x2: $\mathbb{R}$, y1: $\mathbb{R}$, y2: $\mathbb{R}$ }

The region R contains the location ℓ iff $R.x1 \leq \ell.x \leq R.x2$ and $R.y1 \leq \ell.y \leq R.y2$. Thus, the following region includes every point in the plane:

EVERYWHERE := {x1: $-\infty$, x2: ∞ , y1: $-\infty$, y2: ∞ }

The following functions NW, ..., SE : (Location, Region) $\rightarrow$ Region return the intersection of the region passed in with one of the quadrants of a node that was split at the location passed in. For example, NW(m , R) would return the region that includes only the area that is both inside of R and northwest of m (i.e., an x coordinate that is $\leq m.x$ and a y coordinate that is $\leq m.y$).

These functions assume that m falls within the region R , and they are defined as follows:

NW(m , R) := {x1: $R.x1$, x2: $m.x$, y1: $R.y1$, y2: $m.y$ }

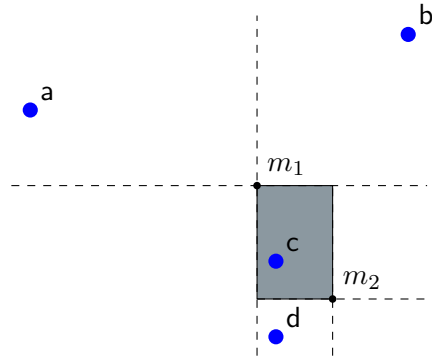
NE(m , R) := {x1: $m.x$, x2: $R.x2$, y1: $R.y1$, y2: $m.y$ }

SW(m , R) := {x1: $R.x1$, x2: $m.x$, y1: $m.y$, y2: $R.y2$ }

SE(m , R) := {x1: $m.x$, x2: $R.x2$, y1: $m.y$, y2: $R.y2$ }

For example, NW(m , R) leaves left side of the region at $R.x1$, but it only allows the region to extend to the right as $m.x$. Any point further to the right than that is outside of the region to the NW of m .

Returning to our example from before



the shaded region, which is the region that lies in the SE quadrant of the split at m_1 and then, within that, the NW quadrant of the split at m_2 , can be calculated as NW(m_2 , SE(m_1 , EVERYWHERE)).

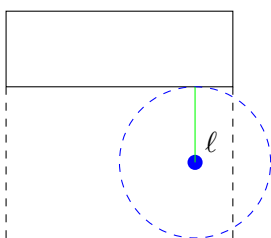
Task 1 – He’s Making a Dist, and Checking It Twice

[8 pts]

We defined a function “dist” before that calculates the distance between two locations. It will also be useful for us to have a function $\text{distTo} : (\text{Location}, \text{Region}) \rightarrow \mathbb{R}$ that calculates the distance from the given location to the *closest* location in the given region. This gives us a *lower bound* on the distance to any location within that region.

To define this function, $\text{distTo}(\ell, R)$, we will break our analysis into a few different cases. The first case is when ℓ is within R . In that case, the distance from ℓ to R is 0.

- (a) Write an expression that is true iff location ℓ falls within the region R .
- (b) If ℓ is not within R , then our next case is when it is directly below, above, to the left, to the right. For example, if ℓ is directly below, then the picture looks like this:

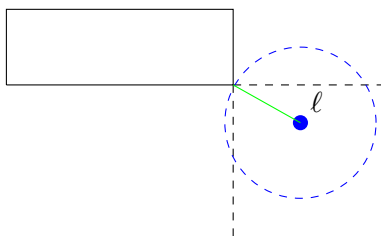


As we can see, the closest location in the region R is directly above ℓ .

Write an expression that is true iff location ℓ is directly below R .

(Note that larger y values are farther **down** on the page.)

- (c) Write an expression that calculates the distance from ℓ to the closest location in R when ℓ is directly below R .
- (d) If ℓ is not within R and not directly below, above, left, or right, then it is in one of the regions diagonally away from R (above and left, above and right, below and left, or below and right). For example, if ℓ is below and right, then the picture looks like this:



In this case, the closest point is the bottom-right corner of R .

Write an expression that is true iff location ℓ is below and right of R .

- (e) Write an expression that calculates the distance from ℓ to the closest location in R when ℓ is below and right of R . Feel free to use the function dist defined before.

Finding the Closest Location

The function $\text{closest} : (\text{LocTree}, \text{Location}, \text{Location}) \rightarrow \text{Location}$ finds the closest location in the given tree to a given location. More specifically, when invoked as $\text{closest}(T, \ell, c)$, it returns either the closest location in the tree T to ℓ or the location c , whichever is closer to ℓ . It is defined as follows:

$$\begin{aligned} \text{closest}(\text{empty}, \ell, c) &:= c \\ \text{closest}(\text{single}(s), \ell, c) &:= c && \text{if } \text{dist}(c, \ell) \leq \text{dist}(s, \ell) \\ \text{closest}(\text{single}(s), \ell, c) &:= s && \text{if } \text{dist}(c, \ell) > \text{dist}(s, \ell) \\ \text{closest}(\text{split}(m, \text{nw}, \text{ne}, \text{sw}, \text{se}), \ell, c) &:= \text{closest}(\text{nw}, \ell, \\ &\quad \text{closest}(\text{sw}, \ell, \\ &\quad \text{closest}(\text{se}, \ell, \\ &\quad \text{closest}(\text{ne}, \ell, c)))) \end{aligned}$$

The case for $\text{split}(\dots)$ performs a tree traversal. It first recurses into the NE quadrant. That call returns the closest location to ℓ within the NE quadrant or, if no location is closer, c . The location returned from the NE quadrant is passed as the third argument to the recursive call into the SE quadrant. That call returns the closest location to ℓ within the SE quadrant or, if no location is closer, the third argument passed to it, which is the closest location from the NE quadrant or, if no location is closer, c .

This process continues through the SW and NW quadrants, so that the location returned from the final recursive call is the closest to ℓ within the NW region or, if none is closer, the closest to ℓ within the SW region or, if none is closer, the closest to ℓ within the SE region or, if none is closer, the closest to ℓ within the NE region or, if none is closer, c . In other words, it returns the closest location within the entire subtree to ℓ or c , whichever is closer.

Note in particular that, if every location in the tree is farther away from ℓ than c is, then closest will return c . (This can be proven formally by induction.) We will need that fact later on!

The particular order chosen here — NE, SE, SW, NW — traverses the quadrants in *clockwise* order. We could traverse the quadrants counter-clockwise or in any order we want by switching the order of $\text{nw}, \dots, \text{ne}$ in the recursive calls.

So far, our closest calculation still calculates the distance between the target location ℓ and every point in the tree. We can fix this by excluding any quadrant that is farther away from ℓ than c is. When this is true, we know immediately that a recursive call to closest on the subtree for that quadrant would return c , so there is no reason to traverse all of its locations. . .

Task 2 – Going Weak in the Trees

[20 pts]

We will define a function $cl : (\text{LocTree}, \text{Location}, \text{Region}, \text{Location}) \rightarrow \text{Location}$ that looks like closest but takes an extra argument that is a region containing all the points in the tree. It will use that region to avoid recursing into quadrants of split nodes when they cannot contain a location closer than c .

More precisely, cl is defined as follows:

$$\begin{aligned} cl(\text{empty}, \ell, R, c) &:= c \\ cl(\text{single}(s), \ell, R, c) &:= c && \text{if } \text{dist}(c, \ell) \leq \text{dist}(s, \ell) \\ cl(\text{single}(s), \ell, R, c) &:= s && \text{if } \text{dist}(c, \ell) > \text{dist}(s, \ell) \\ cl(\text{split}(m, \text{nw}, \text{ne}, \text{sw}, \text{se}), \ell, R, c) &:= c && \text{if } \text{dist}(\ell, c) \leq \text{distTo}(\ell, R) \\ cl(\text{split}(m, \text{nw}, \text{ne}, \text{sw}, \text{se}), \ell, R, c) &:= cl(\text{nw}, \ell, \text{NW}(m, R), && \text{if } \text{dist}(\ell, c) > \text{distTo}(\ell, R) \\ &\quad cl(\text{sw}, \ell, \text{SW}(m, R), \\ &\quad cl(\text{se}, \ell, \text{SE}(m, R), \\ &\quad cl(\text{ne}, \ell, \text{NE}(m, R), c))) \end{aligned}$$

Here, the case for split nodes from closest has become two cases. In the first case, when $\text{dist}(\ell, c) \leq \text{distTo}(\ell, R)$, we know that all the locations within this subtree are farther away than c is from ℓ , so we skip the recursive calls and simply return c . It is only in the second case, when $\text{dist}(\ell, c) > \text{distTo}(\ell, R)$, that we perform the recursive calls.

Note how we are using the functions NW, NE, SW, SE to calculate a smaller region that only includes the part of R that falls within that quadrant and passing that region in the recursive call. (This implements the procedure we described earlier for calculating the region for a given node in the tree.)

- (a) Prove that, if the region R contains all locations in the tree T , then $cl(T, \ell, R, c) = \text{closest}(T, \ell, c)$. Your proof should be by structural induction on T .

Feel free to use the fact that, if R contains all the locations in $\text{split}(m, \text{nw}, \text{ne}, \text{sw}, \text{se})$, then $\text{NW}(m, \text{nw})$ contains all the locations in nw and likewise for ne , sw , and se . (This follows from the representation invariant for split nodes and the definitions of these functions.)

Remember that, because $\text{distTo}(\ell, R)$ is the distance from ℓ to the closest location in R , any other location s in R must satisfy $\text{dist}(\ell, s) \geq \text{distTo}(\ell, R)$. Hence, if $\text{dist}(\ell, c) \leq \text{distTo}(\ell, R)$ ($\leq \text{dist}(\ell, s)$), then we know already that s cannot be closer to ℓ than c . Thus, if R contains all the locations in the tree, then none of them can be closer to ℓ than c , and we can simply return c immediately as the closest, which is exactly what cl does in its definition above.

Hints:

- Do not forget about the section materials! There is a suspiciously similar problem we covered there that you should use as a direct reference for how to organize your proof.
 - Your base case and inductive step will need to contain a proof by cases. Don't forget the to state your cases and write your conclusion for these sub-proofs!
 - Our claim states "if the region R contains all locations in the tree..." but you still need to state that the claim holds in cases when the region R does not contain all the locations.
- (b) Explain, in your words, why your proof idea would still work if we defined cl to traverse the quadrants in counter-clockwise order or any other order we choose.

Homework 9

Due: Saturday, December 7th 11pm

If you have not completed HW9 written. Close this file and finish the written component before starting the coding parts in this worksheet.

Submission

After completing all tasks to follow, submit your solutions on Gradescope. The following completed files should be submitted to **"HW9 Code"**:

locations.ts	locations_test.ts	location_tree.ts	location_tree_test.ts
routes.ts	routes_test.ts	MapView.tsx	

Set up

Start by checking out the starter code using the command

```
git clone https://gitlab.cs.washington.edu/cse331-24au-materials/hw9-campusfriends.git
```

Install the node modules: Create two terminals. Navigate to the client directory in one and the server directory in the other, and run `npm install --no-audit` in both. To run the app, you'll need to run the command `npm run start` in both the client and server directories. Then navigate to where the client is running `http://localhost:8080/`.

Like HW8, mutation is permitted in the /server but not the /client.

Most coding work in industry is making changes to existing code and, most of the time, those changes are being made to add new features or improvements to the software.

In this assignment, we will add a feature to the campus map application we wrote in HW3. You will implement the client and server portions of an application that allows users to identify friends whose walking paths come close to *some point* on this user's path and then to display those on the map and in the legend. The user can then have an opportunity to see their friends during their walk without having to go too far off their usual path.

Rather than having a single page showing the campus map, the starter code now supports multiple users by beginning with a “login” page¹:

Who are you?

Feel free to update the code in `users.ts` to include yourself and your friends.

After logging in, the user sees a menu with three options:

Choose one of these options:

- Update [friends](#)
- Update [schedule](#)
- View [map](#)

The first option takes them to a page where they can select which of the other users are their friends. As we will see below, our new feature will allow some information about their friends schedules to show up when they view the map.

The second option takes them to a page where they can record their schedule:

List each building and the time you move there:

- **10:30:** 331 in CSE
- **11:30:** PHIL 101 in MLR
- **12:30:** lunch in HUB
- **1:30:** office hours in CSE
- **2:30:** 331 in CS2 ([remove](#))

Next class at in named

Saved successfully!

The schedule is recorded as a list of locations where they will be throughout the day. We will assume that the user is at each location until approximately 10 minutes before the next item on the list. During those 10 minutes, they will be walking between the two locations.

The map page now lets them choose a time on their schedule when they walk between locations:

Show path at

 Start at CSE

 End at MLR

Once they do so, the map will show the walk that they will need to make and, below that, a legend showing information about the endpoints of the path.

¹Obviously, a real version of this application would authenticate with a password or similarly secure mechanism. To make this easier to test, we will skip that.

Building on an existing app requires **first understanding existing code**. Know that it is expected that you may have to spend time reading starter code (and tests), using the app, drawing diagrams and/or anything else that's useful to understanding the context around the tasks you'll need to complete. The spec also requires a lot of reading, but its worth it to take time to understand.

Task 3 – Let's Do Dist Thing

[18 pts]

The following types and functions we began working with in HW9 written (some of which we also saw in HW3) have been translated to typescript in `server/src/locations.ts`.

```
type Location = {x: number, y: number};

type Region = {x1: number, x2: number, y1: number, y2: number}

const distance = (loc1: Location, loc2: Location): number
```

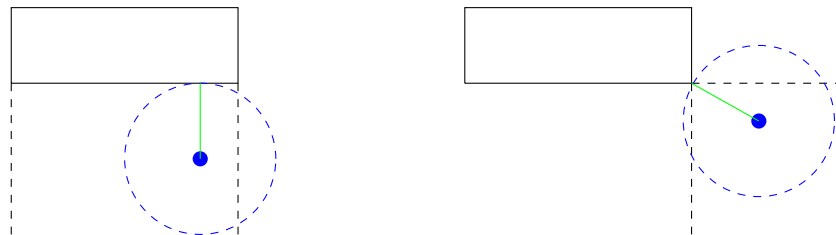
The starter code also includes a function called `squaredDistance` that simply skips the final square root, making it substantially cheaper.

We will start by implementing the following helper function in `locations.ts`:

```
const distanceMoreThan = (loc: Location, region: Region, dist: number): boolean
```

This should check whether the distance from the given location to the *closest* point in the given region is more than `dist`. (An alternative would be to have the function return the distance to the region and let the user compare it to `dist` themselves, but calculating distances often requires a square root. By taking `dist` as an argument, we can answer the “more than” question using `squaredDistance` instead and avoid the need for any square roots.)

It remains to figure out what point in the region is closest to the given point. Since the closest point must be along a boundary, it is either one of the corners or a point along one of the four sides. As the following figures demonstrate, when the location is in line with the region — either vertically or horizontally — then the closest point is the first point along that line, which is on a side. Otherwise, the closest point is the nearest corner.



1. Implement the function `distanceMoreThan` in `locations.ts`. Make sure that your function does not call `distance` or otherwise use any square root calculations.
Be sure to carefully think through your code to make sure it is right. Finding and fixing a bug in this code will only get harder later on, so do the work now to make sure it is right!
2. Write tests for your function in `locations_test.ts`. Make sure that you cover all required cases.
The debugging will only get harder if you miss a bug now.

Task 4 – The Best Things in Life are Tree

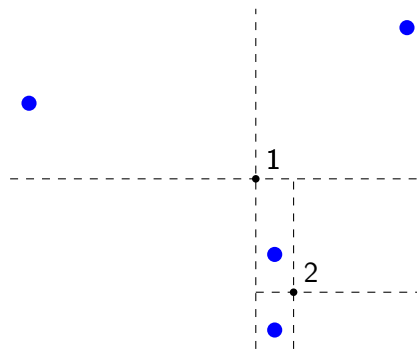
[20 pts]

Recall the definition of `LocTree`, as introduced in the written part, which allows us to efficiently find the closest point to a given location with the recursive structure.

```
type LocTree := empty
           | single(loc : Location)
           | split(at : Location, nw : LocTree, ne : LocTree, sw : LocTree, se : LocTree)
```

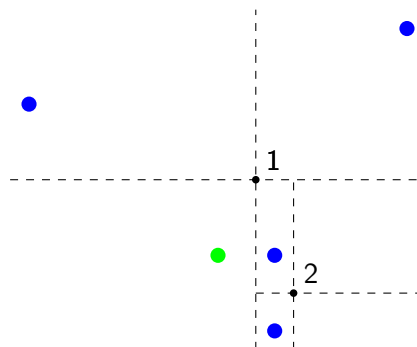
This (and all types and functions we use for this task) is implemented in TypeScript as `LocationTree` in `location_tree.ts`.

As a reminder, to create a tree for the four points below, we would start by splitting them at point “1” (the centroid of those locations). The NW and NE regions then contain only a single point and the SW region is empty. The SE region, however, still contains two points, so we would split them again at point “2” (the centroid of those two), which leaves four regions containing at most one point.



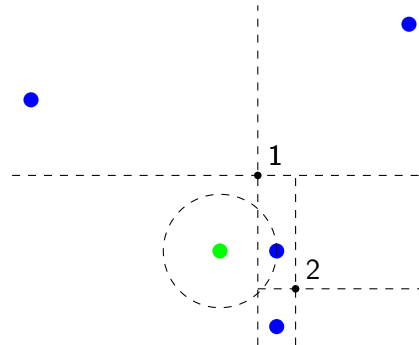
Each node represents some rectangular region in space, but the bounds of that region are not recorded in any field. However, we can easily calculate them bounds as we work down the tree recursively.

We want to be able to find the `Location` in the tree closest to a given `Location`. For example, to find the closest (blue) point of the location tree to the given (green) point.



The root of the tree is the split marked “1”. Since the green node falls into the SW region, we should start by recursively finding the closest point there. That region is empty, so we have no bound on how small the closest point might be.

The region of “1” next closest to the green point is the SE region, so we recursively search there next. That is itself a split node, with the split marked “2” in the picture. The green node is closest to the NW region of “2”, so we search there. That contains a blue node that is at a distance shown by the radius of the circle in this figure:



This distance becomes an upper bound on the closest distance of any point in the tree. We may later find points that are closer, but if they are, they would need to be closer than this region.

The next closest region in the split “2” is the SW region. We can see that its north-west corner falls just within the circle, so it is possible for it to contain a closer node, which means we need to search it recursively. When we do so, we will calculate the distance to the lowest blue point and find that it is farther away. Thus, the first point we found remains the closest, but we have now performed two distance calculations.

The next closest region in the split marked “2” is its NE region, but its distance to the green point is larger than the closest point found so far (the circle does not intersect that region), so we can skip search it all together. The final region of the split marked “2” to consider is the SE region. It, too, does not intersect the circle, so it can be skipped.

That completes the search of the SE region of the split marked “1”. We have a candidate closest node and have performed two distance calculations.

Since we were searching the SE region recursively, when we return, we go back to our search of the split marked “1”. The next closest region (after SW and SE) is the NW region. We find that the circle does not intersect it, so we can skip it. Likewise for the NE region. Thus, we now know for sure that the first point we found was the closest, and we did so using only two distance calculations.

In HW9 written, we introduced the function `cl` which finds the closest Location in a LocTree to a given Location and takes an additional Region argument containing all the Locations in the LocTree which it uses to avoid recursing into quadrants of split nodes that cannot contain a Location closer than the closest location found so far. The recursive search we described above is very similar to this function.

In our implementation here, however, we additionally want to keep track of what steps our code took to find the closest location. Since distance calculations are expensive, rather than returning the closest point, we will return both the point and its distance. (We already calculated it when we determined that it was the closest point, so there is no need to ever calculate it again.) In addition, we will also keep track of the number of distance calculations performed so far in order to make our code more testable. Without that information, we would not be able to tell via tests that our code was actually checking every region in the tree!

To do this, we will change the return value to the following record type that includes all of the information mentioned above:

```
type ClosestInfo = {loc: Location | undefined, dist: number, calcs: number}
```

In addition, rather than keeping track of just the closest `Location` we have found so far, we will instead pass a `ClosestInfo` recording the closest point found so far *and* the number of calculations performed so far between recursive calls.

With that in mind, the actual signature of the function we want to implement is the following:

```
const closestInTree = (tree: LocationTree, loc: Location,  
                      bounds: Region, closest: ClosestInfo): ClosestInfo
```

1. Implement the function `closestInTree` in `location_tree.ts`.

Your function should *begin* by checking whether the distance from `loc` to the bounds of that region is more than the distance to the closest point found so far. If it is, then you can skip searching this subtree. (The closest node remains the one passed in.)

Otherwise, if the tree is empty, then there is nothing to search and the closest node remains the same. If it is a single node, then we will need to perform a distance calculation. If it is closer than the node passed in, then we return it as the closest node; otherwise, we return the node passed in. In either case, we have performed 1 more distance calculation.

Lastly, if we have a split node, then we need to search through the subregions in order by the distance to `loc`. Make a recursive call for each of them, updating the closest node each time to whatever that call returns, and returning the closest node from the last recursive call as the closest node in the subtree.

Note that this step differs from the `cl` definition from HW9 written, which searched the subtrees in counter-clockwise order rather than by distance. The approach we take here will help us find the closest location more efficiently. In HW9 written we completed a proof that `cl` accurately found the closest location, convince yourself now that this alternate order of recursive calls will also be accurate (your answer from Task 2b of the written part may be helpful).

Hint: While it is possible to implement this last case with a loop², it is worth noting that there are, in fact, only *eight* different orders that you might go through the subregions, so you can also implement this with fairly simple `if / else` statements.

2. Write tests for your function in `location_tree_test.ts`. Make sure you cover all required cases. Again, the debugging will only get harder if you miss a bug now.
3. Uncomment the tests for `findClosestInTree` in `location_test_test.ts`. If the function you wrote is correct, these should all pass.

²This would require extra square root calculations. E.g., if the location is in the NW region, then calculating its distance to the SE region involves a square root. The distances to the NE and SW regions, however, do not require square roots!

We now have all the helper functions we need to implement the server part of our new feature.

We want to change the `shortestPath` route so that, in addition to returning the shortest path for the user, it also returns the closest point in the path of any friend that is also walking at that time. We will provide that information in an array of records, each having the following shape:

```
type Nearby = {friend: string, dist: number, loc: Location};
```

1. Modify the function `getShortestPath` in `routes.ts` so that the JSON response sent back includes a list of `Nearby` records for each friend who is walking at the same time.

The starter code already has a loop that identifies the friends who are walking at the same time. You only need to add the code to find the nearest point on the user’s path (`locs`) to any of the points on the friend’s path (`fLocs`).

(Note: If you saw a previous version of the spec, the instructions instead read “You only need to add the code to find the nearest point on their path (`fLocs`) to any of the points on the user’s path (`locs`).” If you implemented this alternative version, that is also okay; you will not be penalized. The instructions now are what we intended them to be, matching examples from our solution we have given.)

If these two lists contain n and m points respectively, then a naive algorithm would perform $n \times m$ distance calculations. Instead, if we put the second list of points into a tree (using `buildTree`) and then call `findClosestInTree` for each point, we can find the closest pair of points with roughly $O(n \log m)$ distance calculations, which is substantially faster for large n and m .

2. Modify the tests this function in `routes_test.ts` so that it also tests the cases where friends are nearby. This will require adding a schedule for another friend and making sure they will be walking between locations at the same hour.

As annoying as it may seem to write these tests, debugging a problem in the UI that is actually a bug in the server is much more work, so be sure that your code is correct before continuing.

Task 6 – Jolly Green Client

[20 pts]

Now that the server is working correctly, we will switch over to the **client**.

We want to change the UI to show all the nearby friends of the user when they are looking at any of the walks they make on campus.

1. Modify `doHourChange` in `MapView.tsx` to record the list of nearby points in the state when it is returned by the server.

We have provided a function called `parseNearbyList` in `nearby.ts` that turns the JSON data back into the TypeScript `Nearby` type.³

2. Modify `renderEndpoints` to draw a circle showing the location of each of the nearby friends and `renderLegendItems` to identify that circle on the legend.

At the bottom of the file, you'll see a commented out list of colors `FRIEND_COLORS` that you are free to use (or you can pick your own colors). If there are more friends than colors, you can either reuse colors or only show that many friends.

Congratulations!! on completing your biggest app of the quarter!

³You're welcome!